SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Integrated Mtech Software Engineering

SWE1002 – Principles of Software Engineering

L10+L11

Faculty In charge:

Prof Shaik Sharmila, Assistant Professor(Research), SCOPE, VIT-AP University

Laundry Management System

24MIS7056 - PAPPURU NIKHILA

24MIS7060 - KARLAPUDI SAI THANUJA

24MIS7064 - KARUMURI CHARAN DEEP REDDY

24MIS7069 - THANGELLAPALLI NIKHITHA

24MIS7070 - JONNALAGADDA SNEHITHA

LIST OF EXPERIMENTS

Ex. No	Title	Page Number
1	1. Problem Analysis - Overview - Scope	4
2	1.Problem Analysis - Objectives - Infrastructure	7
3	2.SRAP - description of individual phase/mode - user characteristics - general constraints	9
4	2.SRAP - Assumptions : dependency - Function requirements - input and output	12
5	3.Data Modelling : Usecases	15
6	3.Data Modelling – Activity	19
7	3.Data Modelling - Sequence	23
8	3.Data Modelling – Class	26
9	3.Data Modelling - Deployment	30
10	3.Data Modelling-Component	33
11	4.Development – Coding	37
12	5.Software Testing - Test Plan	41
Project Presentation with Demo and Report Submission		

Ex.No	**1**
Title	1. Problem Analysis - Overview - Scope

Preparations:

- Identify the real-world issue that needs to be solved.
- Understand the domain of the project and define its boundaries.
- Discuss the overall purpose of developing the Laundry Management System.

Use Case Diagram 1. **Who are Actors?** 2. **Name main Use Cases** 3. What is includes? 4. What is extends? 5. Define good day path.	**Class Diagram** 1. Aggregation 2. Composition 3. Dependency 4. Realization 5. Stereotype 6. Visibility of members
Activity Diagram 1. What activity starts first? 2. What decisions are shown? 3. What are swimlanes used for? 4. What does loop occur? 5. What is end activity?	**Component Diagram** 1. What are main components? 2. What does UI Component do? 3. What connects UI and Database? 4. What is the backend component? 5. How Components communicate?

Questions:

1. In Laundry Management System, identify the main actors — **Student (User)**, **Admin (Laundry Manager)**, and **System Database**. Describe their roles and interactions.
2. What is the relationship between the use cases **Place Laundry Order** and **View Order Status**?
3. What is the relationship between the use cases **Approve Laundry Order** and **Check Pending Orders**?
4. What is the relationship between **Laundry Order** and **User Account**?
5. What is the relationship between **Laundry Record** and **Detergent Type** (e.g., Ariel, Tide, Surf Excel)?
6. What is the relationship between **Laundry Record** and **Order Status Log**?
7. What is the difference between **Order Status** and **User ID**?

8. What is the relationship between **Admin** and the **Laundry Order**?
9. What is the relationship between **Student** and **Hostel/Room Number**?
10. Identify the keys in the database table **Users** – which field can be used as **Primary Key** and which can serve as **Candidate Key**? (Example: UserID → Primary Key; Email → Candidate Key.)
11. What is the relationship between the **Submit Button** and the **Form Validation Event** on the **Place Order** page?
12. What is the relationship between the **Login Form** and the **Buttons (Login/Cancel)** used within it?
13. What is the relationship between the **List of Laundry Orders** and the **Individual Laundry Record (Object)** in programming terms?

Problem Statement

In many hostels and college campuses, managing laundry services is still a manual process involving handwritten records or verbal requests.
This often leads to issues such as misplaced orders, delays in delivery, confusion about washing preferences, detergent stock mismanagement, and lack of proper tracking of completed or pending laundry.
The **Laundry Management System (LMS)** aims to automate this process through a web-based platform.
It allows students to place laundry requests online and enables administrators to manage orders, track progress, update detergent stock, and resolve complaints efficiently.
This ensures smooth communication between students and staff, reduces manual workload, eliminates paperwork, and maintains accurate digital records of all laundry activities.

Overview / Introduction

The **Laundry Management System** is a web application designed to simplify and automate laundry-related operations within hostels and college environments.
It provides two primary roles:

- **User (Student)** – can log in, place laundry orders, check order status, submit complaints, and give feedback.
- **Admin (Laundry Staff/Manager)** – can log in, view and manage all orders, update detergent stock levels, and handle complaints and feedback.

By centralizing all laundry data in a single database, the system eliminates manual record-keeping and ensures transparency between students and administrators. It enhances efficiency by saving time, reducing human error, and providing real-time access to laundry service information.

Scope:

The scope of the **Laundry Management System** includes:

- **Secure login functionality** for both students (users) and administrators.
- **Online laundry order placement** with fields for room number, clothes count, wash type, and detergent selection.
- **Dashboard for students** to track their submitted orders and view their current status (*Pending / Processing / Completed*).
- **Admin interface** to manage all laundry requests, monitor detergent stock levels, and handle complaints or feedback.
- **Centralized database storage** for all orders, complaints, feedback, and stock information for record-keeping and future analysis.

Limitations (Out of Current Scope)

- **Automatic email or SMS notifications** for order updates.
- **Online payment integration** for service billing.
- **Real-time delivery tracking** or pickup scheduling system.

These features are reserved for future enhancement *(as noted in Appendix C: TBD List)*.

Conclusion:

This experiment provides a complete analysis of the problem and forms the foundation for the **Laundry Management System**.
It clearly defines the objectives and identifies all the functional and technical requirements necessary for developing and implementing the project efficiently.
The system ensures smooth management of laundry services, effective tracking of orders, complaint resolution, and stock monitoring.
Overall, it establishes a structured framework that enhances service quality, reduces manual effort, and improves operational efficiency in hostel and college laundry operations.

EXP	**2**
Title	1.Problem Analysis - Objectives - Infrastructure

Preparations:

- Study the user requirements and existing system.
- Analyze the needs of both admin and student modules.
- Identify the tools and technologies required to implement the system effectively

Problem Analysis:

In many hostels and college campuses, laundry services are still managed manually using paper slips, registers, or direct verbal communication between students and laundry staff. This traditional approach creates several challenges and inefficiencies:

1. **Delays and Miscommunication:**
   Students must personally visit the laundry area to submit their clothes or inquire about their status, leading to unnecessary waiting and confusion.
2. **Human Errors:**
   Manual record-keeping often results in misplaced laundry, wrong delivery, or incorrect entry of student details such as room numbers or item counts.
3. **Lack of Transparency:**
   Students cannot easily track the progress or completion status of their laundry. Similarly, staff members find it difficult to monitor pending or completed orders.
4. **Stock and Resource Mismanagement:**
   Keeping track of detergent stock (e.g., Ariel, Tide, Surf Excel) and other consumables manually can lead to shortages or overstocking due to the absence of accurate data.
5. **Poor Complaint and Feedback Handling:**
   When issues arise—such as missing clothes or delayed service—students have no proper channel to raise complaints or provide feedback. This results in dissatisfaction and unaddressed service quality problems.

Proposed Solution :

To overcome these issues, a **Laundry Management System (LMS)** is proposed. This web-based solution automates the entire process of **laundry order placement, tracking, stock monitoring, and feedback management** through a centralized and user-friendly interface.

The system provides two main roles:

- **Student (User):** Can place laundry orders, check order status, submit complaints, and give feedback.
- **Administrator (Laundry Manager/Staff):** Can view and update all orders, manage detergent stock, handle complaints, and monitor service performance.

By digitizing laundry operations, the proposed system ensures:

- Faster and more accurate service.
- Transparent communication between students and laundry staff.
- Efficient stock management and record-keeping.
- Reduced manual workload and errors.

Objectives:

The main objectives of the Laundry Management System are:

1. To automate the process of placing, tracking, and managing laundry service requests.
2. To provide a structured and organized system for maintaining order, stock, and service records.
3. To improve transparency and communication between students (users) and administrators (laundry managers).
4. To minimize manual paperwork and reduce errors in order tracking and record maintenance.
5. To enable quick and easy access to order status, complaints, and feedback history.
6. To design a responsive, browser-based interface that is accessible across multiple devices.
7. To streamline hostel or campus laundry operations through efficient digital workflow management.

The purpose of the **Laundry Management System** is to replace the traditional manual methods of managing hostel or institutional laundry services with a digital, automated solution.

Infrastructure:

1. Hardware Requirements:

- Processor :Intel i5 or above 8GB Ram
- Hard Disk: Minimum 256GB
- Display: Standard Resolution monitor.

2. Software Requirements:

- Operating Systems:Windows10 or above
- Front-end: HTML,CSS ,JavaScript.

3. Back End: java
4. Database: My SQL Server
5. Browser Support: Google chrome, Mozilla Firefox

Conclusion:

This experiment covers the detailed analysis of the existing laundry management problem, defines the purpose of developing the system, and specifies the exact tools, technologies, and environment required.

It provides a solid foundation for the next stage — Software Requirement Analysis and Planning (SRAP) — which describes the modules, user characteristics, and functional requirements of the system.

EXP	3
Title	2.SRAP - description of individual phase/mode - user characteristics - general constraints

Preparations:

- Study the requirement specification document (SRS).
- Identify each functional module of the Laundry Management System.
- Analyze the roles of students interacting with the system.
- Note the constraints and limitations identified during requirement analysis

Software Requirements Analysis and Planning:

Description of Individual Modules:

The Laundry Management System is divided into the following functional modules based on the SRS:

1. **Login Module:**
   This module provides secure access for both students and administrators. Users must log in using valid credentials, ensuring that only authorized individuals can access their respective dashboards. It verifies user details against the database before granting access.
2. **Student Dashboard:**
   The student dashboard offers an organized interface where users can place laundry orders, check order status, file complaints, and provide service feedback. It helps students track the progress of their laundry and manage all related actions conveniently.
3. **Order Placement Module:**
   This module allows students to book laundry services by selecting detergent type, number of clothes, and service type (e.g., normal or express). Once submitted, the order details are saved in the database and made available for admin review and processing.
4. **Admin Dashboard:**
   The admin dashboard enables the administrator to manage all laundry operations, including viewing new orders, updating order status, managing detergent stock, and reviewing complaints or feedback submitted by students. It provides a complete overview of daily activities.
5. **Order Management Module:**
   Used by the administrator to view, update, or change the status of laundry orders. Admins can mark orders as “Processing,” “Completed,” or “Delivered.” Students can instantly view these updates on their dashboards, ensuring transparency.
6. **Stock and Complaint Management Module:**
   This module maintains detergent stock levels and complaint records. Admins can adjust stock quantities and receive alerts for low inventory. Students’ complaints are logged and stored for admin review and resolution, promoting smooth service operations.

7. **Logout Module:**
   Ends the current user session securely and redirects the user to the login page. It ensures that personal data remains protected and prevents unauthorized access after logout.

Each of these modules interacts with the central MySQL database, ensuring all data is consistent, updated, and securely maintained.

User Characteristics:

User Characteristics

The system is designed for two categories of users: **students** and **administrators**.
• **Students** are the primary users who utilize the system to manage their laundry-related activities. They are expected to have basic computer knowledge and the ability to operate a web browser. Each student is provided with valid login credentials to access their personalized account. Within their dashboard, they can place new laundry requests, check the progress or status of existing orders, file complaints if necessary, and review their previous service history for reference.

• **Administrators** (or hostel laundry staff/managers) have higher-level access rights and play a supervisory role in the system. They are responsible for managing and monitoring all laundry operations, including order verification, detergent stock management, and complaint resolution. Administrators can also update the status of laundry requests and maintain accurate records within the database. To perform these operations efficiently, administrators should be familiar with basic data entry, data management, and system navigation.

Both categories of users require a computer or mobile device with an active internet connection and a modern web browser. The interface has been intentionally designed to be simple, intuitive, and user-friendly, ensuring that even users with minimal technical skills can easily access and operate the system.

General Constraints:

The Laundry Management System functions under a defined set of operational constraints established during the requirement analysis. Access to the system is limited strictly to authenticated users, ensuring that unauthorized individuals cannot log in or view any service details. Each student can place laundry orders, submit complaints, or give feedback only through their personal account, maintaining data privacy. Administrative privileges are exclusively granted to the admin, who alone has the authority to update order statuses, manage stock levels, and handle complaints or feedback within the system.

From a performance standpoint, the system should be capable of handling multiple concurrent user sessions without noticeable delay. Database transactions such as inserting or updating laundry records must execute quickly to maintain responsiveness.

Security is also a major constraint. User passwords are to be stored securely, and all sensitive pages are protected from unauthorized access. The system must log out inactive sessions automatically to prevent misuse.

Regarding software and hardware limitations, the system requires a machine with at least an Intel i5 processor and 8 GB RAM, running on Windows 10 or above, as specified in the SRS. It uses the XAMPP server environment, combining Apache, PHP, and MySQL for development and execution. The application should run smoothly on common browsers such as Google Chrome or Mozilla Firefox.

Finally, operational constraints include the need for basic user training and stable connectivity when the system is deployed on a local or network server.

Conclusion:

This experiment defines the detailed module structure of the Laundry Management System, explains the responsibilities and characteristics of each user type, and describes the major constraints under which the system will function. The outcome of this analysis serves as the basis for the next stage, which will focus on system assumptions, dependencies, and functional input-output design.

EXP	4
Title	2.SRAP - Assumptions : dependency - Function requirements - input and output

Preparations:

Before proceeding to the design and implementation phase, certain assumptions and dependencies must be recognized to ensure that the system functions as intended. It is also important to identify all the core functional requirements of the system and to define how input and output data will be handled at the user and system level.

Software Requirements Analysis and Planning :

Assumptions & Dependencies:

During the development of the Laundry Management System, several assumptions were made to simplify the design and ensure clarity of scope.

It is assumed that all students and administrators who use the system have access to a stable computer or laptop with a proper internet connection. Each user is expected to possess valid login credentials issued by the organization. The organization's network is assumed to be secure, ensuring that no unauthorized individuals gain access to the system.

It is also assumed that the administrator has the authority to view, edit, and accept student laundry requests according to hostel policy. The number of employees using the system is moderate, so the database will not face heavy performance loads in its initial implementation.

The system depends on a few key components for its operation. It relies on the Apache web server (through XAMPP), PHP for server-side scripting, and MySQL for database management. The functioning of the system depends on proper installation and configuration of these components. Additionally, browser compatibility is a dependency; the system has been tested on Google Chrome and Mozilla Firefox, and it assumes that users will access the application using one of these browsers.

The system also depends on the integrity of data stored in the database. If the database connection fails or is corrupted, the system's ability to process or retrieve laundry records will be affected. Hence, regular database backups are recommended.

Functional Requirements:

The functional requirements describe what the Laundry Management System must do to achieve its intended purpose.

1. **User Authentication:** The system must provide a secure login interface for both students and administrators. Each user must enter a valid username and password to access their respective dashboards. This ensures that only authorized users can manage or request laundry services.
2. **Laundry Order Management:** Students must be able to place laundry orders by selecting the detergent type, quantity of clothes, and service type. Once submitted, the order details are stored in the database and marked as pending until processed by the administrator.
3. **Order Processing and Update:** Administrators must be able to view all pending laundry orders and update their status to "Processing" or "Completed." The status change should be reflected in the database and visible to the respective student on their dashboard.
4. **Complaint and Feedback Handling:** Students must be able to submit complaints related to their orders and provide feedback about the laundry service. Administrators can view, respond to, or resolve these complaints to maintain service quality and user satisfaction.
5. **Data Storage and Management:** All details, including user profiles, laundry orders, complaints, feedback, and detergent stock levels, must be securely stored in the MySQL database. Proper access control should be implemented to prevent unauthorized data modification.
6. **Stock Monitoring and Alerts:** The system should maintain detergent stock records and automatically alert the administrator when the stock level of any detergent brand falls below the predefined limit. This ensures smooth operation and timely restocking.
7. **Logout Functionality:** The system must allow both students and administrators to securely log out after their session, ensuring that no unauthorized access occurs after use and that sensitive data remains protected.

These requirements ensure that the Laundry Management System operates efficiently and securely, providing a reliable platform for handling orders, complaints, and stock information with proper authentication and data integrity.

Inputs and Outputs:

The Laundry Management System handles several types of input and output data throughout its operation.

Input:

- The primary inputs are provided by users through the web interface. Students input their login credentials on the login page and later enter details such as room number, number of clothes, detergent type, and washing preferences while placing an order. They can also provide input when submitting complaints or giving feedback. The administrator provides input while logging in, updating stock levels, changing order statuses, and responding to complaints or feedback.
- All input data is validated to ensure completeness and correctness. The system checks for empty fields, invalid room numbers, and incorrect date or quantity formats. Incomplete or invalid entries trigger error messages, ensuring accurate data collection and preventing incorrect transactions or stock updates.

Output:

- The system outputs are displayed through a clean and responsive web dashboard. After a successful login, students view their personalized dashboard showing order details, laundry status (e.g., “Pending,” “In Process,” or “Completed”), and confirmation messages after submitting an order, complaint, or feedback. Status updates are reflected automatically to keep the student informed.
- For administrators, the system outputs include real-time data on all active orders, detergent stock levels, complaints, and feedback summaries. The output also generates confirmation messages when stock is updated or an order status is changed. All outputs are simultaneously reflected in the database, ensuring that the displayed information remains accurate and synchronized across every module.

These inputs and outputs form the core of user-system interaction, ensuring smooth and accurate communication between the user interface and the database.

Conclusion:

This experiment defines the underlying assumptions and dependencies that guide the design and functioning of the Laundry Management System. It also details the functional requirements and describes how data is input, processed, and displayed to users. The clarity gained from this analysis ensures that future design and implementation phases proceed with precision and consistency.

EXP	5
Title	3.Data Modeling : Usecases

Preparation:

Before creating the use case model, it is necessary to understand the functional flow of the system from a user's perspective. Each actor's interaction with the system must be identified and represented clearly in the use case diagram. This helps define how the system behaves under different conditions and ensures that every functional requirement is captured visually.

Data Modelling: Use Case

Introduction:

Use case modeling is an important aspect of the system analysis phase. It helps in identifying the various actors (users interacting with the system) and the specific use cases (functionalities they perform).

For the **Laundry Management System (LMS)**, the main purpose of the use case diagram is to represent how students and administrators interact with the system to carry out different operations such as placing laundry orders, tracking status, managing complaints, and monitoring detergent stock.

This model provides a clear picture of the overall workflow of the laundry service, defining each user's access level and the system's response to their actions. It also acts as a foundation for designing further models such as the system architecture and sequence diagrams in the later stages of development.

Actors in the System:

The Laundry Management System has two main actors:

1. Student (User): The student uses the system to log in, place laundry orders, check the status of their submitted requests, and provide feedback or file complaints regarding the service. They can also view past order history to keep track of completed and pending laundry services

.

2. Administrator (Manager / Staff): The administrator is responsible for managing all laundry operations. They can view and update laundry orders, monitor detergent stock levels, handle complaints and feedback from students, and maintain records of daily laundry activities. Each user interacts with the system through a dedicated interface, with access permissions defined according to their role.

Major Use Cases:

The core use cases identified from the SRS for the Laundry Management System are as follows:

1. **Login:**
   Both students and administrators log in to the system using their respective credentials to access personalized dashboards.
2. **Place Order:**
   The student submits a new laundry request by entering details such as the number of clothes, detergent preference, and delivery time.
3. **View Order Status:**
   The student can check the current processing stage of their submitted laundry orders — whether pending, in progress, or completed.
4. **View Past Orders:**
   The student can view a detailed history of all previously submitted and completed laundry orders for reference.
5. **Update Detergent Stock:**
   The administrator updates the detergent quantities available in the system to ensure sufficient stock for daily operations.
6. **View Orders:**
   The administrator or manager reviews all active, pending, or completed laundry orders to manage daily workloads efficiently.
7. **Assign Staff:**
   The manager assigns specific staff members to handle collection, washing, or delivery tasks for each laundry order.
8. **Handle Complaints:**
   The administrator reviews complaints submitted by students, takes corrective actions, and updates their resolution status.
9. **Manage Orders:**
   The administrator updates order progress, marks completion, and ensures that each service cycle is accurately recorded.
10. **Logout:**
    All users — students, managers, and administrators — securely log out from the system after completing their operations.

Each use case represents a specific interaction between the actor and the system that fulfills part of the overall functionality.

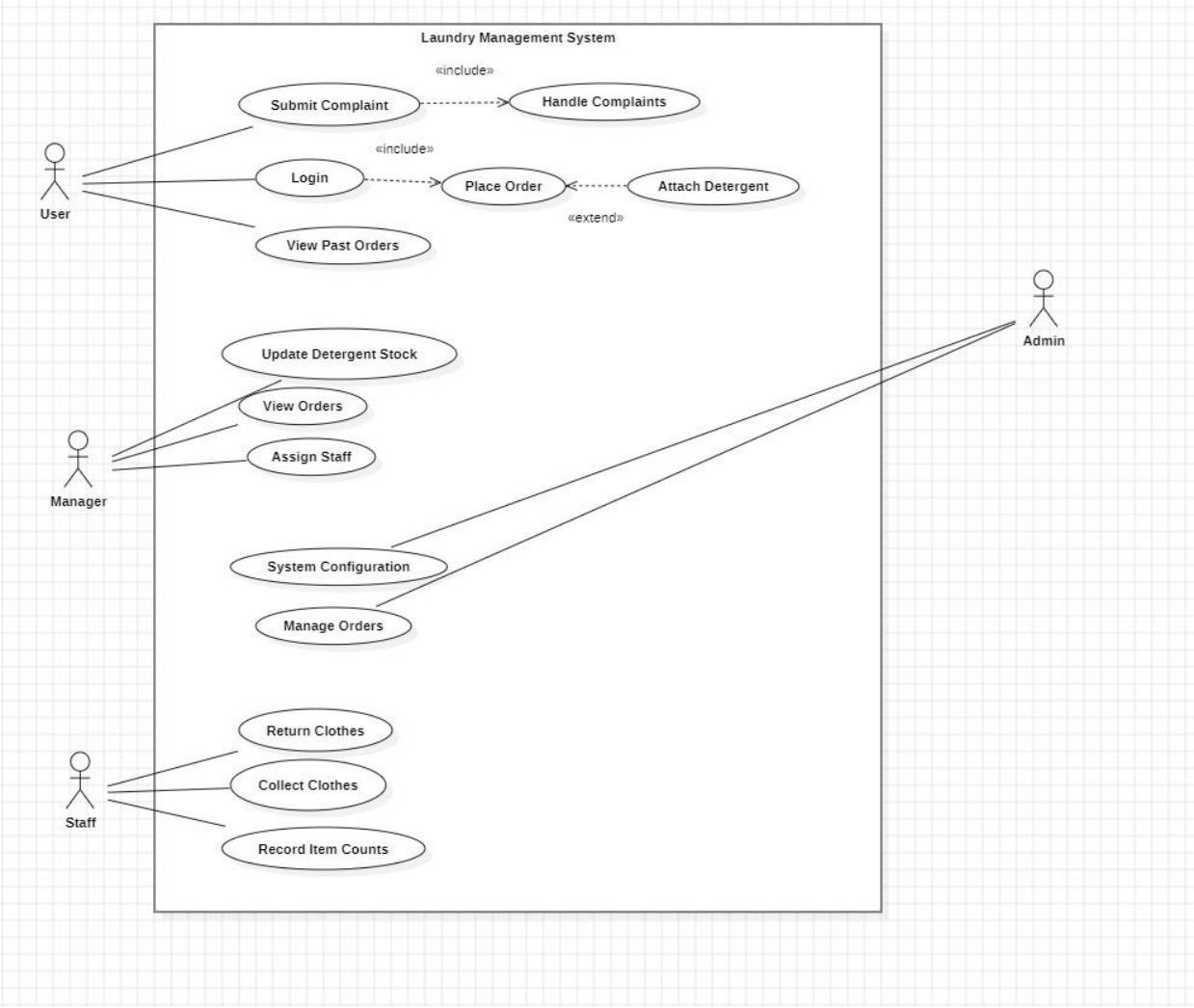
Explanation of Relationship:

From the diagram, we can observe that both the User (Student) and Admin must perform a Login operation before accessing any further use cases.
Once logged in, the **User** can perform actions such as **Placing Laundry Orders**, **Filing Complaints**, **Giving Feedback**, and **Viewing Order Status**.
The **Admin**, on the other hand, can **Manage Laundry Orders**, **Update Detergent Stock**, **View and Resolve Complaints**, and **Check Feedback Reports** to monitor overall service quality.
Both the User and Admin use cases are connected to the **Database**, which serves as a shared central resource for storing and retrieving all laundry-related data. The database ensures that every action performed by the Admin, such as updating stock or changing order status, is immediately reflected on the User's dashboard, thereby maintaining real-time synchronization and accuracy throughout the system.

Conclusion:

This experiment successfully defines and illustrates all use cases of the Laundry Management System. The diagram and description together explain how each actor interacts with the system and how the data flows among the user, admin, and database. This model forms the base for further diagrams such as the activity, sequence, and class models that detail the internal behavior of each use case.

Diagram:


Laundry Management System
«include»
Submit Complaint
Handle Complaints
«include»
Login
Place Order
Attach Detergent
«extend»
View Past Orders
User
Admin
Update Detergent Stock
View Orders
Assign Staff
Manager
System Configuration
Manage Orders
Return Clothes
Collect Clothes
Record Item Counts
Staff

EXP	6
Title	3.Data Modeling – Activity

Preparations:

Before designing the activity diagram, it is important to understand how data flows between the system's modules.

The developer must identify the starting point, sequence of user actions, system responses, and the end conditions for the process.

This diagram visualizes the internal flow of control from login to logout and shows how different activities connect to achieve a complete complaint process.

Data Modelling – Activity:

Introduction:

An activity diagram represents the dynamic behavior of the system and illustrates how control flows between different activities. It captures the step-by-step process of how users and administrators interact with the system.

In the **Laundry Management System**, the activity diagram illustrates two main workflows:

1. The **student's flow** — placing a laundry order, tracking its status, filing complaints or feedback, and viewing previous laundry history.
2. The **administrator's flow** — managing all incoming laundry requests, updating order progress, handling complaints and feedback, and monitoring detergent stock levels to ensure smooth service operation.

Description of Activtes:

The system begins when the user selects their role. There are two main roles — **Student (User)** and **Admin**.
If the **Student** role is selected, the system moves to the Student Login activity. The student enters their credentials, and the system verifies them. If the login is successful, the student gains access to the **Student Dashboard**; otherwise, an invalid login message appears, prompting them to retry until correct details are entered.

Once logged in, the student can choose among four main actions:

• **Place Laundry Order** – The student fills in details such as room number, number of clothes, detergent type, and wash preference. The system submits this data to the backend and stores it in the database as a new order with the status marked as *Pending*.
• **Check Order Status** – The student can view the current status of their submitted laundry orders (e.g., Pending, In Process, or Completed). The system retrieves and displays the relevant information from the database.
• **Submit Complaint** – The student can file a complaint related to delay, damage, or service issues. The complaint is

recorded in the system for admin review.
• **Provide Feedback** – After receiving their clothes, the student can give feedback or ratings to help improve the service.

After completing these actions, the student can securely log out, ending the active session and returning to the initial home page.

On the **Admin** side, when the Admin role is selected, the process continues with **Admin Login**. The system validates the admin's credentials, and if incorrect, it displays an error message asking for re-entry. After successful authentication, the admin is directed to the **Admin Dashboard**.

From here, the admin can perform various management actions:

• **View All Orders** – The system fetches all laundry requests submitted by students, showing their current status and details.
• **Update Order Status** – The admin selects any order and updates its status as *Processing* or *Completed* based on progress. The system then reflects this change in the database.
• **Manage Stock Levels** – The admin can monitor and update detergent stocks such as Ariel, Tide, or Surf Excel. The system displays low-stock alerts when any detergent quantity falls below a threshold.
• **View Complaints and Feedback** – The admin reviews complaints submitted by students and provides resolutions, as well as analyses overall feedback for service improvements.

After completing all necessary tasks, the admin can also log out, which safely ends the session and returns the system to its initial idle state.

Flow Explanation:

The activity diagram begins with the **Start Node**, followed by a **decision node** that determines whether the actor is a **student user** or an **admin**.

In the **Student Path**, the system first verifies the login credentials. If the credentials are valid, the student is redirected to the **dashboard**; otherwise, the system prompts a retry. Once logged in, the student can choose to **place a laundry order**, **check order status**, **submit a complaint**, or **give feedback**. Each of these activities interacts with the database to update or retrieve relevant information. After completing any task, the user is redirected back to the dashboard, allowing multiple operations within the same session. Finally, when the student logs out, the activity flow for the user ends.

In the **Admin Path**, after successful authentication, the admin gains access to the **admin dashboard**, where they can **view and manage laundry orders**, **check detergent stock**, **review complaints**, and **analyze feedback**. When viewing orders, the admin can **update their status** as "Processing" or "Completed," and changes are saved in the database. If detergent levels are low, the admin can **update stock quantities** accordingly. Once all administrative actions are completed, the admin logs out, marking the end of the session.

Both the student and admin flows **merge at the End Node**, indicating that all activities conclude with the **logout process**, completing the system's operational cycle.

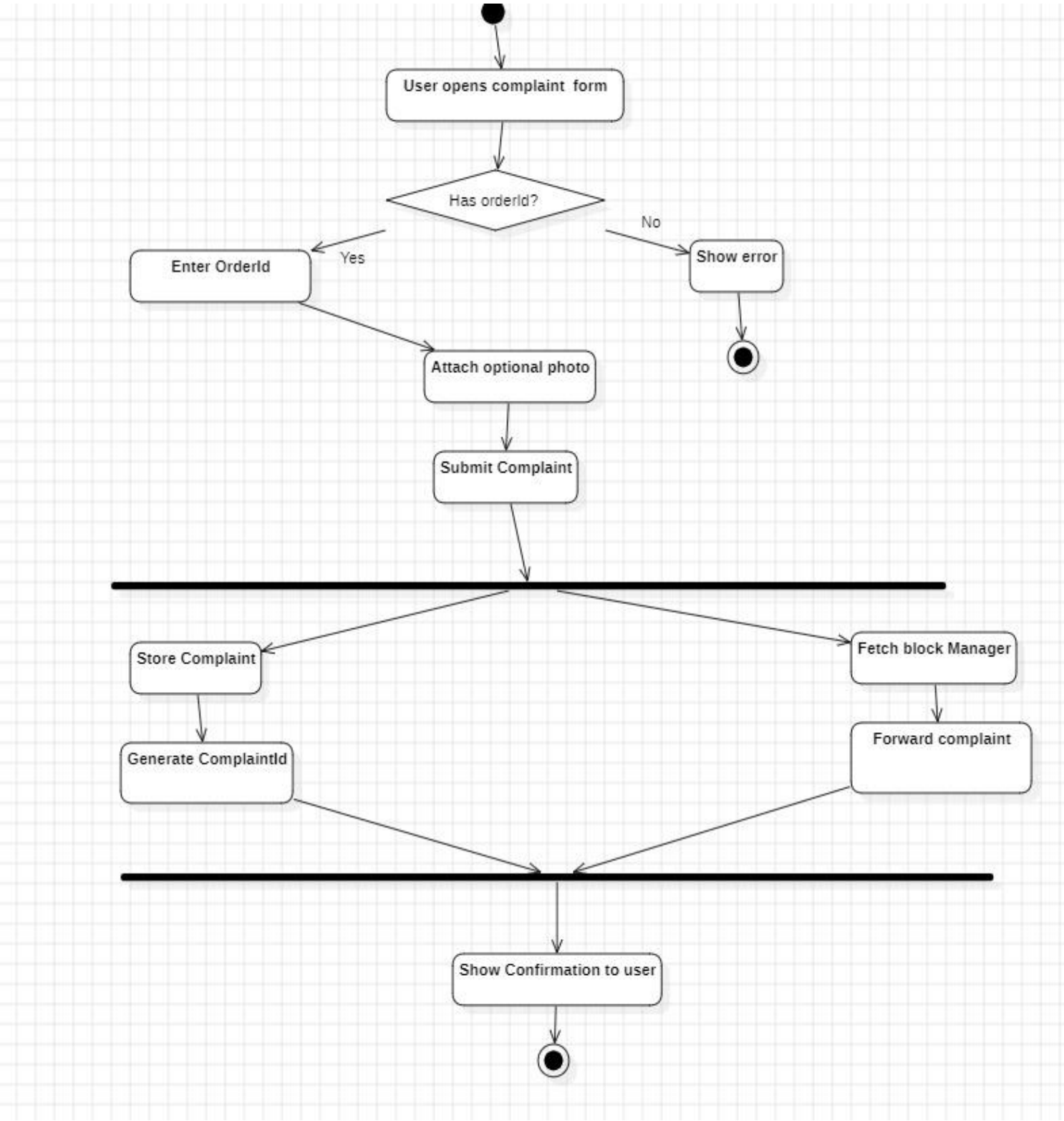
Explanation of Diagram:

The activity diagram represents the **complaint submission process** in the Laundry Management System. It starts when the **user opens the complaint form**. The system checks if the user has a valid **Order ID**. If not, an error message is displayed, ending the process. If the Order ID is valid, the user can **attach an optional photo** as supporting evidence. The user then proceeds to **submit the complaint**, after which the process splits into two concurrent actions. One branch **stores the complaint details** in the database and **generates a unique Complaint ID**. The other branch **fetches the respective block manager** and **forwards the complaint** for review and action. Once both actions are completed, the system **shows a confirmation message** to the user, indicating that the complaint has been successfully recorded and forwarded for resolution. The process then ends with the **final node**, marking completion of the complaint workflow.

Conclusion:

The activity diagram of the **Laundry Management System Complaint Module** clearly illustrates the flow of control between the user and the admin components. It shows how a user submits a complaint by entering an order ID, optionally attaching a photo, and how the system processes and forwards the complaint to the respective block manager. By depicting the sequence of storing complaints, generating unique IDs, and notifying the manager, the diagram provides a clear understanding of the complaint-handling workflow. This representation helps visualize the interaction between the user and backend services, forming a strong foundation for developing detailed **sequence diagrams** and **data flow diagrams** in subsequent design phases.

Diagram:


User opens complaint form
Has orderId?
Yes
No
Enter OrderId
Show error
Attach optional photo
Submit Complaint
Store Complaint
Fetch block Manager
Generate ComplaintId
Forward complaint
Show Confirmation to user

EXP	7
Title	3.Data Modeling – Sequence

Preparations:

Before constructing the sequence diagram, it is essential to identify all major interactions between the user, the administrator, the system interface, and the database. The objective is to represent the chronological flow of messages exchanged among these components during different operations of the Laundry Management System.

Data Modelling - Sequence:-

Introduction:

A sequence diagram illustrates how processes interact with one another and the order of communication between them over time. It focuses on message flow between actors and system components.

In the Laundry Management System, the sequence diagram illustrates the key activities such as user login, placing laundry orders, admin login, updating detergent stock, managing complaints, and tracking the status of orders. Each of these interactions follows a systematic flow of operations, beginning from data input by the user to processing and database update or retrieval, ensuring smooth coordination between the frontend interface and backend system for efficient laundry service management.

Description of Flow:

- The diagram begins with the **User Login and Order Placement** interaction. The user opens the laundry order form through the LMS Web UI, and the system first validates the user session by communicating with the authentication service. If the session is valid, the user is allowed to proceed; otherwise, the system prompts for re-login or displays an authentication error.
- Once logged in, the **user submits a new laundry order** by entering required details such as clothing type, quantity, and detergent preference. The system validates this data and forwards the order request to the Order Service. Upon successful creation, the order is stored in the database, and the system calculates the pickup time and service schedule.
- The **Order Service** then generates an order confirmation and sends it back to the LMS Web UI, where the user is shown a confirmation message indicating that the order has been successfully placed.
- Simultaneously, the **Notification Service** is triggered to alert the laundry manager of the new order. The service sends a “New Order Notification” to the Manager Dashboard and also triggers an SMS notification to inform the manager about the pending pickup.
- Once the **Manager Dashboard** receives the notification, it updates the list of active orders for processing. The manager is then able to view, assign, or manage the new order accordingly.

- Finally, the system sends a notification back to the user confirming that the manager has been notified and the pickup process is scheduled. This ensures seamless coordination between user, order, and manager modules, providing efficient and transparent laundry service management.

Explanation of Interaction:

The diagram consists of six main lifelines — the **User**, **LMS Web UI**, **Auth Service**, **Order Service**, **Notification Service**, and **Manager Dashboard**.
Each operation follows a structured flow:

1. The **user** initiates an action, such as opening the order form or submitting a laundry request.
2. The **LMS Web UI** validates the session through the **Auth Service** to ensure the user is authenticated.
3. Once verified, the **Order Service** processes the request, performs necessary calculations like pickup scheduling, and records the order details in the database.
4. After successful order creation, a confirmation message is sent back to the user interface, displaying the success status.
5. Simultaneously, the **Notification Service** notifies the **Manager Dashboard** about the new order and sends an alert or SMS to the manager for prompt action.

This interaction flow ensures that every order request is validated, recorded, and communicated efficiently between all system components.
The alternate conditions (alt frames) in the diagram represent possible variations, such as **session validation failure**, **order submission errors**, or **notification delays**, making the system more reliable, responsive, and fault-tolerant.

Behavioural View:

The behavioral representation of this diagram shows how user and admin activities are time-ordered and dependent on system feedback.

The system operates as a mediator between the actors and the database, ensuring proper validation, authentication, and synchronization.

Each message exchange occurs sequentially, meaning that no action progresses until the previous one is acknowledged.

This approach guarantees reliability, consistency, and data integrity throughout all interactions.

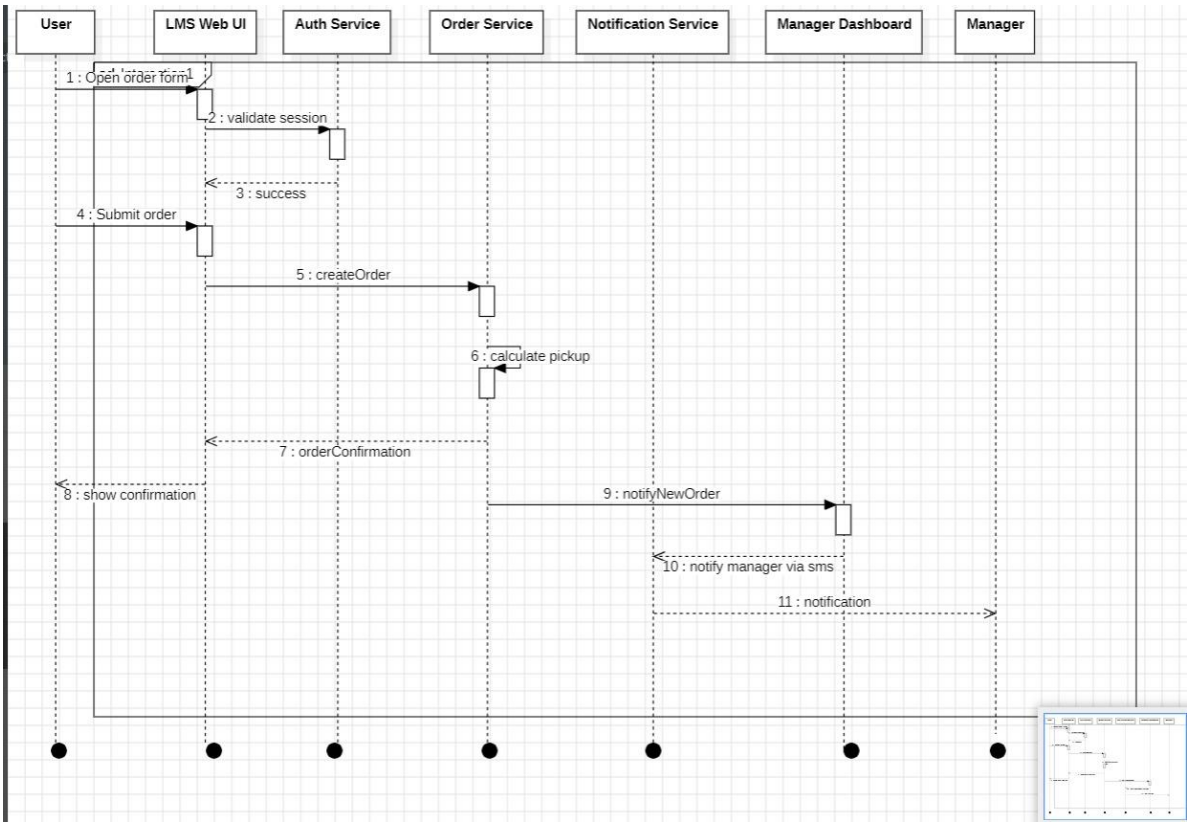
Conclusion:

The sequence diagram of the Laundry Management System clearly demonstrates the flow of control between the user, administrator, system, and database.

It represents how messages are passed for login validation, order submission, collecting , and delivering clothes in a sequential and logical manner.

This diagram provides a detailed understanding of how the system behaves dynamically over time and forms a strong foundation for system implementation and testing.

Diagram:



EXP	8
Title	3.Data Modeling – Class

Preparation:

Before designing a class diagram, it is important to identify all the entities that make up the system and their relationships with one another. A class diagram helps define the structure of the system by showing classes, their attributes, operations, and associations. This representation is vital for transforming the conceptual design into object-oriented code during implementation.

Data Modelling Class:

Introduction:

A class diagram is a static structure diagram that describes the classes of the system and the relationships among them. It forms the backbone of object-oriented modeling by defining what data the system holds and how it behaves.

In the Laundry Management System, the class diagram defines the main classes involved in managing laundry operations — namely **User**, **Admin**, and **LaundryOrder**. Each class contains its own attributes (such as user details, order ID, detergent type, and status) and operations (methods) that represent the actual workflow of the system. The relationships between these classes describe how users place orders, track their laundry, and give feedback, while the admin monitors and updates order progress, handles complaints, and manages detergent stock levels stored in the system.

Description of Classes:

In the Laundry Management System, the first class in the diagram is the **User** class. It represents the hostel student or customer who interacts with the system to place laundry orders and track their progress. The **User** class includes attributes such as *userID*, *name*, and *roomNumber* to uniquely identify each user and link them with their orders. The main method of this class, *login()*, ensures secure system access for each user.

The second class is the **Manager** class, which represents the person responsible for supervising laundry operations and staff. The **Manager** class contains attributes such as *managerID*, *block*, and *contactNumber* for identification and communication purposes. The key operations of this class include *viewOrders()* and *assignStaff()*, which allow the manager to oversee ongoing orders and assign staff members to specific tasks.

The third major class is the **Order** class, which acts as the central component of the system, linking users, staff, and laundry details. It stores information such as *orderID*, *orderDate*, *pickupDate*, *returnDate*, *washType*, *detergentChoice*, *items*, and *status*. Its main method, *calculateCounts()*, helps determine the number of clothing items and other related metrics. The **Order** class is associated with other supporting classes like **ClothingItem**, **OrderStatus**, **Complaint**, **WashType**, and **DetergentType**, making it the core of the workflow.

The **Staff** class represents the workers who handle the laundry process. Attributes such as *staffID*, *name*, and *assignedOrders* identify each staff member, while methods like *collectClothes()*, *recordCounts()*, and *returnClothes()* define their operational roles in managing laundry items and updating order progress.

Additional classes such as **DetergentStock**, **DetergentType**, and **WashType** help maintain inventory and categorize washing preferences. The **Complaint** and **ComplaintStatus** classes manage user grievances and their resolution stages, ensuring proper communication between users and the management.

Together, these classes represent the complete workflow of the laundry management process — from placing an order and assigning staff to updating detergent stocks and handling complaints — ensuring smooth coordination and efficient service delivery across the system.

Explanation of Relationships:

The class diagram shows clear associations between the classes. The User class is connected to the Order class to indicate that one user can place multiple laundry orders. Similarly, the Manager class is associated with both the Order and Staff classes, showing that the manager can oversee multiple orders and assign them to different staff members for processing.

These associations are crucial to represent how real-world entities interact within the laundry system. The Staff class is linked to the Order class to indicate that one staff member can handle several orders, such as collecting, washing, and returning clothes. The Order class is also connected with other supporting classes like Complaint, WashType, DetergentType, and OrderStatus to maintain detailed records of each order's progress, washing method, and detergent used.

The cardinality between these classes — such as one-to-many relationships for User–Order, Manager–Staff, and Staff–Order — implies that multiple orders can belong to a single user, several staff members can be managed by one manager, and each staff member can process multiple laundry requests. These associations collectively ensure efficient data flow and coordination throughout the Laundry Management System.

Overall Design Explanation:

This class diagram effectively models the structure of the Laundry Management system in object-oriented terms. The encapsulation of attributes and operations within each class ensures modularity, making the design easy to maintain and extend.

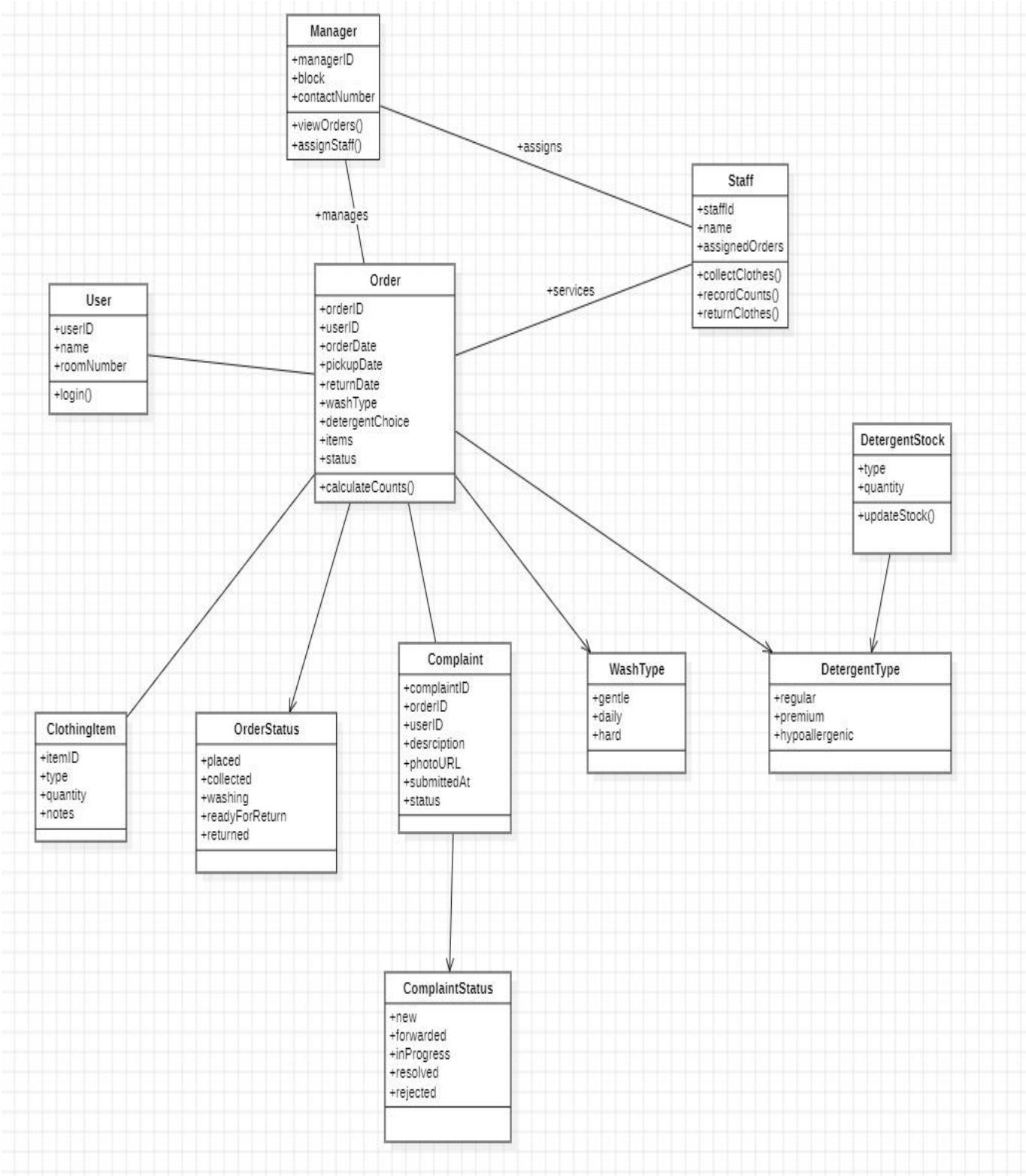
Each class performs a well-defined role — the Student acts as the requester, the Manager acts as the controller or decision-maker, and the Order Request serves as the data handler and link between both actors.

This design clearly defines system responsibilities and lays the foundation for the implementation of object-oriented code in languages like Java, PHP, or Python.

Conclusion:

The Class Diagram of the **Laundry Management System** illustrates the logical structure and object relationships of the application. It clearly defines how users, managers, and staff interact with laundry orders, detergent stock, and complaints through a set of attributes and methods. This model serves as a blueprint for the implementation phase, ensuring that all system components are well-structured, reusable, and easy to maintain, thereby supporting efficient management of laundry operations.

Diagram:


Manager
+managerID
+block
+contactNumber
+viewOrders()
+assignStaff()
+assigns
Staff
+staffId
+name
+assignedOrders
+collectClothes()
+recordCounts()
+returnClothes()
+manages
Order
+orderID
+userID
+orderDate
+pickupDate
+returnDate
+washType
+detergentChoice
+items
+status
+calculateCounts()
+services
User
+userID
+name
+roomNumber
+login()
DetergentStock
+type
+quantity
+updateStock()
Complaint
+complaintID
+orderID
+userID
+desrciption
+photoURL
+submittedAt
+status
WashType
+gentle
+daily
+hard
DetergentType
+regular
+premium
+hypoallergenic
ClothingItem
+itemID
+type
+quantity
+notes
OrderStatus
+placed
+collected
+washing
+readyForReturn
+returned
ComplaintStatus
+new
+forwarded
+inProgress
+resolved
+rejected

EXP	9
Title	3.Data Modeling – Deployment

Preparation:

Before creating a deployment diagram, it is important to understand how the physical components of the system are distributed across the hardware environment.

A deployment diagram represents the hardware configuration, the software components deployed on each node, and the communication links between them.

For the Laundry Management System , the deployment model includes the Student, Web Server, and Database Server nodes that together make the application operational.

Data Modelling Deployment:

Introduction:

A deployment diagram is used to describe the physical arrangement of hardware and software components in a system.

It illustrates how the system's modules are deployed on various nodes and how they interact with each other through network connections.

In the Laundry Management System, the deployment diagram illustrates how the user interface, application logic, and database are structured and interact with each other. It represents a three-tier architecture consisting of the Client, Application Server, and Database Server, ensuring efficient data flow, scalability, and secure communication. The client layer handles user interactions through a responsive web interface, the application server manages business logic and service requests, while the database server stores and retrieves information related to orders, complaints, feedback, and detergent stock.

Description of Deployment Architecture:

The deployment of the Laundry Management System follows a three-layered architecture that ensures efficiency, scalability, and secure communication among various components.

1. Client Node:

- The client node represents the end user's device, such as a computer or smartphone, through which the application is accessed via a web browser or mobile app.
- It provides the **User Interface** that allows users, managers, admins, and staff to log in, place laundry orders, view order status, manage stock, and handle complaints or feedback.
- The client communicates with the web server using **HTTPS requests**, ensuring secure transmission of data between the user device and the application server.

2. Web Server Node:

- The web server acts as the middleware between the client and the database, hosting the **Laundry Management Application**.
- It processes incoming requests, executes business logic, and interacts with the database for reading or updating records.
- The web server includes three primary modules:
  - **Authentication Module:** Validates login credentials for all roles (user, admin, staff, manager) and ensures secure access control.
  - **Order & Service Management Module:** Handles laundry order placement, pickup and return scheduling, complaint registration, feedback collection, and detergent stock monitoring.
  - **Administrative Module:** Enables the admin and manager to oversee service progress, update stock, and manage system-level updates or user accounts.
- The web server ensures efficient execution of operations and communicates with the database using **SQL queries** through secure server-side processing (Java/JSP).

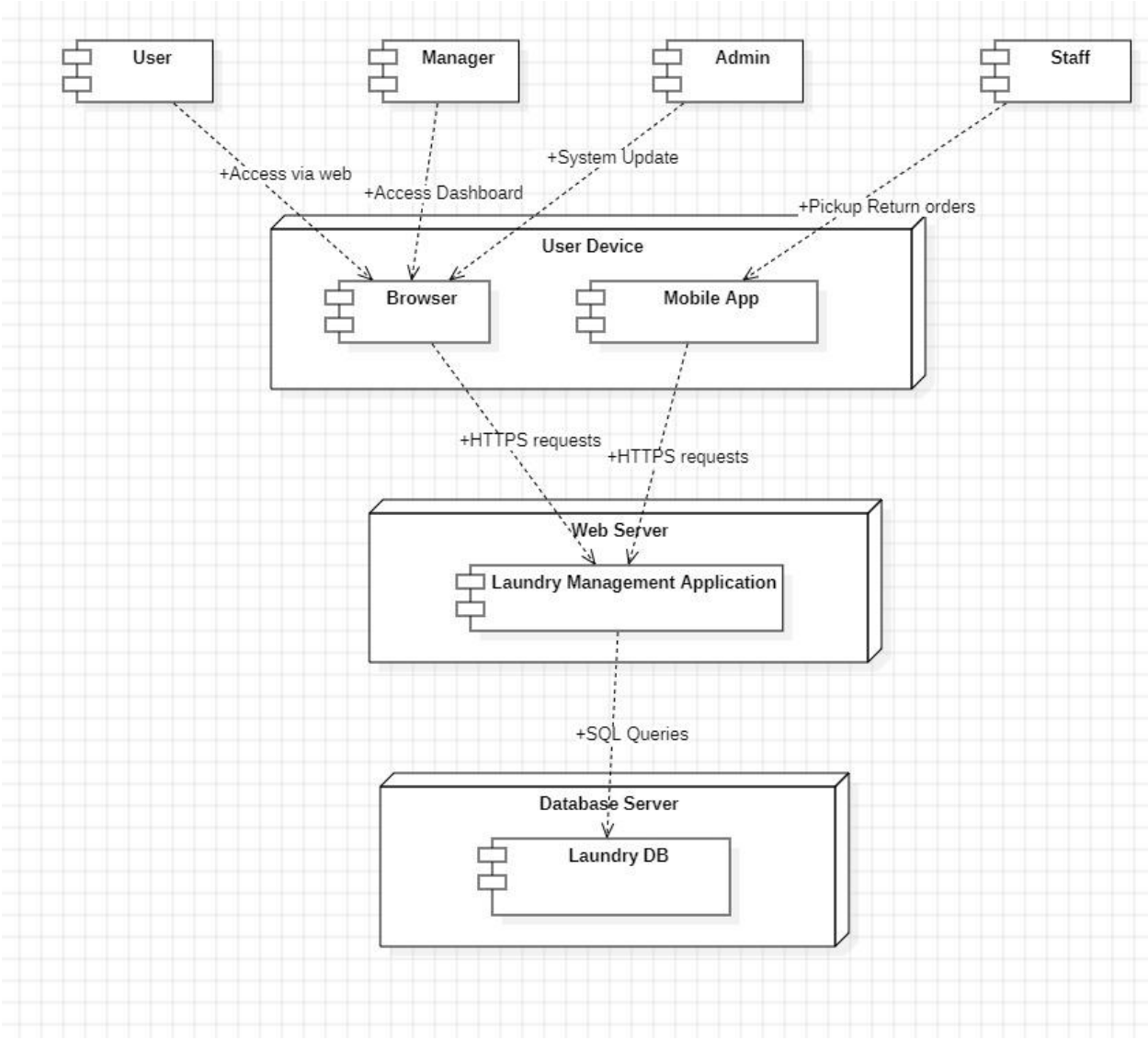
3. Database Server Node:

- The database server is responsible for storing and managing all persistent data used in the system.
- It contains several key tables, including:
  - **User Table:** Stores login credentials and user profiles for students, staff, managers, and administrators.
  - **Order Table:** Maintains records of laundry orders with details such as order ID, clothes count, detergent type, and order status.
  - **Complaint Table:** Records all user complaints along with their resolution status.
  - **Feedback Table:** Stores user feedback and ratings to improve service quality.
  - **Stock Table:** Tracks detergent stock levels and generates alerts when quantities fall below threshold levels.
- The web server connects to the database using a **secure SQL connection**, ensuring reliable data retrieval and updates while maintaining data integrity and confidentiality.

Conclusion:

The Deployment Diagram of the Laundry Management System provides a clear visualization of how the system's components are physically organized and how communication occurs between different nodes.
It follows a structured three-tier architecture that separates the presentation, application, and data layers to enhance scalability, performance, and maintainability.
This architecture ensures smooth interaction between users, the web server, and the database, forming a secure, efficient, and reliable foundation for the overall operation of the Laundry Management web application.

Diagram:


User
Manager
Admin
Staff
+Access via web
+Access Dashboard
+System Update
+Pickup Return orders
User Device
Browser
Mobile App
+HTTPS requests
+HTTPS requests
Web Server
Laundry Management Application
+SQL Queries
Database Server
Laundry DB

EXP	10
Title	3.Data Modeling – Component

Preparation:

A component diagram is used to show how the system is organized into smaller functional units called components.

Each component represents a specific part of the software such as modules, interfaces, or services.

In the Laundry Management System, the application is structured into clearly defined components to ensure modular design, easy maintenance, and scalability.

The major components include the User Interface, Web Server Modules, and Database Components.

Data Modelling – Component:

Introduction:

A component diagram is used to show how the system is organized into smaller functional units called components.

Each component represents a specific part of the software such as modules, interfaces, or services.

In the Laundry Management System, the application is structured into clearly defined components to ensure modular design, easy maintenance, and scalability.

The major components include the User Interface, Web Server Modules, and Database Components.

Description of Components:

The Laundry Management System consists of three main component layers that interact seamlessly to handle all the system's operations:

1. Client Component (Web Interface):

- This component serves as the front-end of the system, accessible to both students and administrators through a web browser.
- It is developed using **HTML**, **CSS**, and **JavaScript**, providing a responsive and user-friendly interface.
- Students can log in, place laundry orders, check order history, file complaints, and give feedback through interactive forms.
- Administrators access the same web interface to monitor service activities, track detergent stocks, and manage staff.
- All actions performed in this layer are sent to the backend server for processing via HTTP requests.

2. Web Server Components:

- The web server acts as the **core logic layer**, implemented using **Java**. It processes all user and admin requests and interacts with the database for data retrieval and updates.
- It includes several major functional modules:
  - **Authentication Module:** Handles secure login and user verification. It validates credentials before granting access to either the student or admin dashboard.
  - **Order Management Module:** Processes student laundry orders, updates order status, and manages the overall workflow from order placement to completion.
  - **Complaint Management Module:** Manages complaint submissions, allows the admin to track and resolve issues, and stores complaint records in the database.
  - **Inventory Management Module:** Tracks detergent stock levels (like Ariel, Tide, Surf Excel) and alerts the admin when any item falls below a predefined limit.
  - **Staff Management Module:** Handles staff details, assignments, and workload distribution for efficient service management.
  - **Notification Service:** Sends real-time updates to users regarding order confirmation, status changes, and complaint resolutions.
- These modules collectively ensure smooth communication between the web interface and the database, maintaining consistent system performance.

3. Database Component:

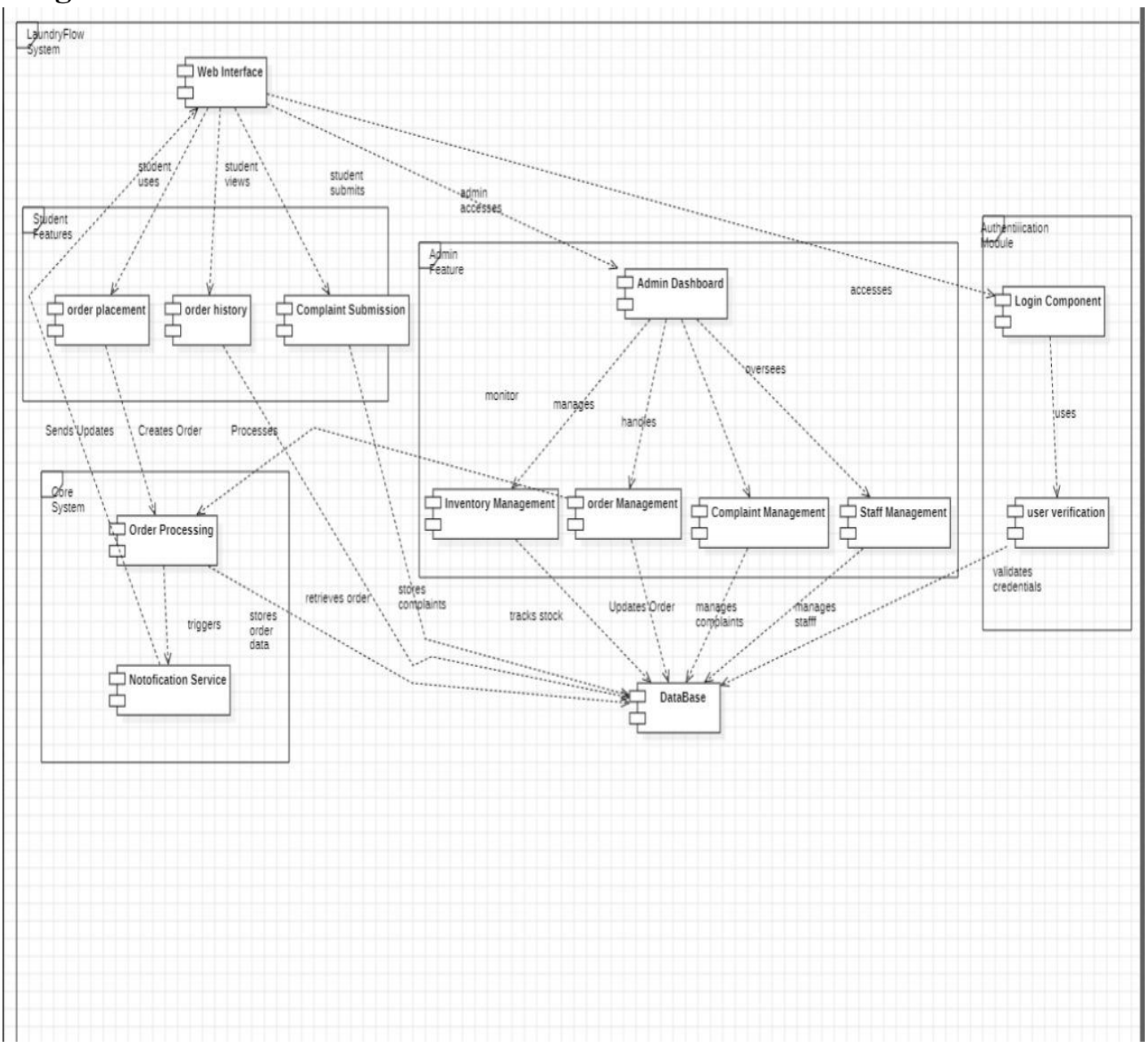
- The **Database** forms the system's data storage layer and handles all CRUD (Create, Read, Update, Delete) operations required for laundry management.
- It is implemented using **MySQL**, ensuring secure and structured data handling.
- The database includes the following key tables:
  - **User Table:** Stores login credentials and profile details for both students and administrators.
  - **Order Table:** Maintains records of all laundry orders, including order ID, detergent used, order status, and timestamps.
  - **Complaint Table:** Keeps track of all submitted complaints along with their current status and resolution details.
  - **Feedback Table:** Contains feedback and ratings provided by students to help improve the service quality.
  - **Stock Table:** Manages detergent quantities and updates inventory levels automatically when an order is processed.
- All database interactions are securely managed through JDBC connections from the Java backend, ensuring data consistency and reliability.

Each of these components works together to create a fully functional and automated laundry management ecosystem. The client interface ensures ease of use, the web server provides intelligent processing, and the database guarantees reliable storage and retrieval, forming an efficient and scalable LaundryFlow System.

Conclusion:

The **Component Diagram of the LaundryFlow System** clearly depicts how the application is structured into three major layers — **Client Interface**, **Web Server Components**, and **Database Server**.
Each module performs a specific and well-defined function, ensuring seamless coordination between user interaction, business logic, and data management.
This layered architecture promotes **modularity, scalability, and maintainability**, allowing easy updates or expansion in the future.
Overall, the design supports **efficient laundry service automation**, secure data handling, and a user-friendly experience for both students and administrators.

Diagram:


LaundryFlow System
Web Interface
student uses
student views
student submits
admin accesses
Student Features
order placement
order history
Complaint Submission
Admin Feature
Admin Dashboard
accesses
Authentication Module
Login Component
monitor
manages
handles
oversees
uses
Sends Updates
Creates Order
Processes
Core System
Order Processing
Inventory Management
order Management
Complaint Management
Staff Management
user verification
validates credentials
retrieves order
stores complaints
tracks stock
Updates Order
manages complaints
manages staff
triggers
stores order data
Notofication Service
DataBase

EXP	11
Title	4.Development - Coding

Preparation:

Before going to exp 11 revisiting Software Requirements Specification and noting all mentioned functionalities , technology used .

Development -Coding:

The **Laundry Management System** uses **JavaScript** to handle all the interactive and dynamic operations — including login management, order handling, complaints, feedback, and stock control.

1. Role Selection and Login Logic

This section manages whether the user logs in as a **Student** or **Admin**. It toggles UI visibility and stores the current role.

```javascript
let currentRole = null;
let orders = [
  { id: 1, name: "John Doe", room: "101", clothes: "5", wash: "Gentle Wash", detergent: "Ariel", status: "Pending" },
  { id: 2, name: "Jane Smith", room: "205", clothes: "8", wash: "Daily Wash", detergent: "Tide", status: "Processing" },
  { id: 3, name: "Robert Johnson", room: "312", clothes: "12", wash: "Hard Wash", detergent: "Surf Excel", status: "Completed" }
];
let complaints = [
  { id: 1, orderId: "2", text: "My clothes came back with a stain that wasn't there before.", status: "Pending" }
];
let feedbacks = [
  { id: 1, text: "Great service! My clothes always come back fresh and clean." }
];
let detergentStock = {
  ariel: 15,
  tide: 8,
  surf: 22
};

function selectRole(role) {
  document.querySelectorAll('.role-option').forEach(option => {
    option.classList.remove('active');
  });

  if (role === 'student') {
    document.querySelectorAll('.role-option')[0].classList.add('active');
    document.querySelector('.login-btn').textContent = 'Login as Student';
  } else {
    document.querySelectorAll('.role-option')[1].classList.add('active');
```

```
    document.querySelector('.login-btn').textContent = 'Login as Admin';
  }
}

function login() {
  let roleElements = document.querySelectorAll('.role-option');
  let role = 'student'; // default

  if (roleElements[1].classList.contains('active')) {
    role = 'admin';
  }

  currentRole = role;
  document.querySelectorAll(".container, .login-container").forEach(div => div.classList.add("hidden"));

  if (role === "student") {
    document.getElementById("studentDashboard").classList.remove("hidden");
  } else if (role === "admin") {
    document.getElementById("adminDashboard").classList.remove("hidden");
    updateStockDisplay();
  }
}
```

2. Order Management Logic

This part handles placing orders, displaying order lists, and linking them with detergent stock levels.

```
let orders = [
  { id: 1, name: "John Doe", room: "101", clothes: "5", wash: "Gentle Wash", detergent: "Ariel", status: "Pending" },
  { id: 2, name: "Jane Smith", room: "205", clothes: "8", wash: "Daily Wash", detergent: "Tide", status: "Processing" },
  { id: 3, name: "Robert Johnson", room: "312", clothes: "12", wash: "Hard Wash", detergent: "Surf Excel", status: "Completed" }
];
```

```
function placeOrder() {
  const name = document.getElementById("orderName").value.trim();
  const room = document.getElementById("roomNumber").value.trim();
  const clothes = document.getElementById("clothesCount").value;
  const wash = document.getElementById("washType").value;
  const detergent = document.getElementById("detergentType").value;

  if (!name || !room || !clothes) {
    alert("Please fill all details!");
    return;
  }

  // Check detergent availability
  const detergentKey = detergent.toLowerCase().replace(" ", "");
  if (detergentStock[detergentKey] <= 0) {
    alert(`Sorry, ${detergent} is out of stock. Please choose another detergent.`);
    return;
  }

  // Reduce stock
  detergentStock[detergentKey]--;
  updateStockDisplay();

  const order = {
    id: orders.length + 1,
    name,
    room,
```

```
    clothes,
    wash,
    detergent,
    status: "Pending"
  };
  orders.push(order);
  alert("Order placed successfully!");
  document.getElementById("orderName").value = "";
  document.getElementById("roomNumber").value = "";
  document.getElementById("clothesCount").value = "";
  showPage("statusPage");
  displayOrders();
}
```

3. Stock Management Logic

The **Admin** uses this section to view and modify detergent inventory in real time.

```
function updateStockDisplay() {
  document.getElementById('arielStock').textContent = detergentStock.ariel;
  document.getElementById('tideStock').textContent = detergentStock.tide;
  document.getElementById('surfStock').textContent = detergentStock.surf;

  // Add low stock warning
  Object.keys(detergentStock).forEach(detergent => {
    const element = document.getElementById(`${detergent}Stock`);
    if (detergentStock[detergent] <= 5) {
      element.classList.add('stock-low');
    } else {
      element.classList.remove('stock-low');
    }
  });
}

function updateStockEditDisplay() {
  document.getElementById('arielStockEdit').textContent = detergentStock.ariel;
  document.getElementById('tideStockEdit').textContent = detergentStock.tide;
  document.getElementById('surfStockEdit').textContent = detergentStock.surf;
```

```
  // Add low stock warning
  Object.keys(detergentStock).forEach(detergent => {
    const element = document.getElementById(`${detergent}StockEdit`);
    if (detergentStock[detergent] <= 5) {
      element.classList.add('stock-low');
    } else {
      element.classList.remove('stock-low');
    }
  });
}

function updateStock(detergent, change) {
  if (detergentStock[detergent] + change >= 0) {
    detergentStock[detergent] += change;
    updateStockEditDisplay();
    updateStockDisplay();
  }
}
```

4. Complaint and Feedback Logic

This module allows students to submit complaints or feedback, which admins can later view.

```
function submitComplaint() {
  const id = document.getElementById("complaintOrderId").value.trim();
  const text = document.getElementById("complaintText").value.trim();
  if (!text) return alert("Please enter your complaint.");
  complaints.push({ id: complaints.length + 1, orderId: id || "N/A", text, status: "Pending" });
  alert("Complaint submitted successfully!");
  document.getElementById("complaintOrderId").value = "";
  document.getElementById("complaintText").value = "";
  backToDashboard();
}

function submitFeedback() {
  const text = document.getElementById("feedbackText").value.trim();
  if (!text) return alert("Please enter feedback!");
  feedbacks.push({ id: feedbacks.length + 1, text });
  alert("Thank you for your feedback!");
  document.getElementById("feedbackText").value = "";
  backToDashboard();
}
```

```
function displayAdminComplaints() {
  const adminList = document.getElementById("adminComplaintList");
  adminList.innerHTML = complaints.length
    ? complaints.map(c => `
      <div class="complaint-item">
        <strong>Complaint #${c.id}</strong><br>
        Order ID: ${c.orderId}<br>
        📝 ${c.text}<br>
        ⚙ Status: ${c.status}
      </div>`).join("")
    : "<p>No complaints found.</p>";
}

function displayAdminFeedbacks() {
  const adminList = document.getElementById("adminFeedbackList");
  adminList.innerHTML = feedbacks.length
    ? feedbacks.map(f => `
      <div class="feedback-item">
        💬 ${f.text}
      </div>`).join("")
    : "<p>No feedback found.</p>";
}
```

5. Navigation and Display Management

Handles smooth transitions between login, student, and admin views without reloading pages.

```
function showPage(pageId) {
  document.querySelectorAll(".container").forEach(div => div.classList.add("hidden"));
  document.getElementById(pageId).classList.remove("hidden");
  if (pageId === "statusPage") displayOrders();
  if (pageId === "adminOrdersPage") displayAdminOrders();
  if (pageId === "adminComplaintsPage") displayAdminComplaints();
  if (pageId === "adminFeedbackPage") displayAdminFeedbacks();
  if (pageId === "stockManagementPage") updateStockEditDisplay();
}

function backToDashboard() {
  document.querySelectorAll(".container").forEach(div => div.classList.add("hidden"));
  if (currentRole === "student") {
    document.getElementById("studentDashboard").classList.remove("hidden");
  } else if (currentRole === "admin") {
    document.getElementById("adminDashboard").classList.remove("hidden");
    updateStockDisplay();
  } else {
    document.getElementById("loginPage").classList.remove("hidden");
  }
}
```

EXP	11
Title	4.Development - Testplan

Preparation:

Before going to exp12 we need to know all about types of testing like black box testing , white box testing.

Development – Testplan:

Introduction:

Test Plan

The test cases for the Laundry Management System are designed across various levels and types of testing to ensure that the application performs accurately, efficiently, and reliably under all conditions. Each testing level focuses on validating different functional and integration aspects of the system as described below:

1. **Unit Testing:**
   • This level of testing is conducted on individual modules such as user authentication, order placement, feedback submission, and stock update to verify that each component works correctly in isolation. Each Java function and frontend script is tested independently to ensure accurate input handling and output generation.

2. **Integration Testing:**
   • This stage validates the interaction between interconnected modules, such as the connection between the Order Management Module and the Database or the synchronization between the Complaint and Feedback modules. It ensures that data flow between the frontend (HTML/JS) and backend (Java) components functions smoothly without communication errors.

3. **System Testing:**
   • The complete Laundry Management System is tested as a unified application to verify that all integrated modules perform according to the defined functional requirements. End-to-end workflows such as login, placing orders, updating stock, and resolving complaints are tested to ensure overall reliability and compliance with system specifications.

4. **Acceptance Testing:**
   • This final phase is performed to confirm that the system meets the expectations of both students and administrators. It checks whether all functional, usability, and performance requirements are satisfied, ensuring the application is ready for real-time deployment and day-to-day use in a hostel or institutional environment.

Each test case is documented with its respective preconditions, input data, expected result, actual result, and postconditions. This systematic approach ensures full traceability of testing outcomes and maintains high quality standards throughout the development and validation phases of the Laundry Management System.

Test Plan

S.No	Test Case ID	Test Case Description	Precondition	Test Data	Expected Result	Postcondition	Actual Result	Status	Comments
1	TC_01	Verify student login with valid credentials	Student account must exist in system	Username: student01, Password: stud@123	Student successfully logs into the dashboard	Student session created and redirected to dashboard	As expected	Pass	Login verified successfully
2	TC_02	Verify student login with invalid credentials	User enters incorrect credentials	Username: student01, Password: wrong@123	System displays "Invalid username or password" message	No session created	As expected	Pass	Error message displayed correctly
3	TC_03	Verify order placement by student	Student must be logged in	Clothes: 10, Detergent: Ariel, Wash Type: Normal	Laundry order submitted successfully	Order record stored in Orders table	As expected	Pass	Order recorded successfully
4	TC_04	Verify admin login with valid credentials	Admin account must exist	Username: admin, Password: admin@123	Admin redirected to Admin Dashboard	Admin session created	As expected	Pass	Admin login verified
5	TC_05	Verify admin can update detergent stock	Admin must be logged in	Detergent: Tide, Quantity: +10	Stock quantity updated successfully in database	Updated stock visible in stock table	As expected	Pass	Stock update working correctly

6	TC_06	Verify low stock alert message	Admin logged in, stock below threshold	Detergent: Surf Excel = 3 units	System displays "Low stock! Please restock soon" alert	Low-stock warning displayed on dashboard	As expected	Pass	Alert displayed as expected
7	TC_07	Verify student can file a complaint	Student must be logged in	Complaint: "Clothes delayed for 2 days"	Complaint recorded successfully in Complaints table	Complaint ID generated and stored	As expected	Pass	Complaint module verified
8	TC_08	Verify feedback submission by student	Student must be logged in	Feedback: "Service was fast and good."	Feedback submitted successfully	Feedback entry stored in Feedback table	As expected	Pass	Feedback form working correctly

Note:

- Test cases to be defined based on levels and types of test:
  - Unit Test
  - Integration Test
  - System Test
  - Acceptance Test etc.,